Bloking Day-4

PRAKTIK PEMROGRAMAN C#

- Nama: Yohanes Adriano Putra
- Kelas: XI PPLG 1
- SMK Prestasi Prima

Tujuan Praktikum

Tujuan praktikum:

- Menerapkan variabel, tipe data, kontrol alur, dan method dalam C#.
- Menganalisis dan memperbaiki kesalahan logika pada kode.
- Merancang algoritma untuk mekanisme game.
- Memahami konsep OOP seperti inheritance dan abstraction.
- Menganalisis serta mengoptimalkan kode yang tidak efisien.

Tools yang digunakan:

- Unity Hub & Unity Editor
- Visual Studio Code / Visual Studio
- C#

Tantangan 1
Inventory

Kesalahan Pertama terdapat pada:

if (items.Count <= maxSlot)

Seharusnya:

if (items.Count < maxSlot)

Alasan:
Jika maxSlot = 10, kondisi <= masih memungkinkan item ditambahkan ketika jumlah item sudah 10. Akibatnya inventory dapat memiliki 11 item, melebihi kapasitas maksimal.

```
using System.Collections.Generic;
using UnityEngine;

public class Inventory : MonoBehaviour
{
    public int maxSlot = 10;
    private List<string> items = new List<string>();

    public bool AddItem(string itemName)
    {
        if (items.Count < maxSlot)
        {
            items.Add(itemName);
            return true;
        }

        return false;
    }

    public void RemoveItem(string itemName)
    {
        items.Remove(itemName);
    }

    public List<string> GetItems()
    {
        return new List<string>(items);
    }
}
```

Kesalahan Kedua terdapat pada:

List<string> result = items;
return result;

Seharusnya:

return new List<string>(items);

Alasan:
result hanya menunjuk ke list asli. Jika list yang dikembalikan diubah dari luar, data inventory asli juga dapat ikut berubah.

Tantangan 2
Damage System

Aturan sistem damage:

- Fire mengalahkan Earth → ×1,5
- Earth mengalahkan Water → ×1,5
- Water mengalahkan Fire → ×1,5
- Elemen yang lemah → ×0,5
- Elemen sama → ×1
- Critical → ×2
- Critical memiliki peluang 20%
- Defense dikurangi setelah multiplier
- Damage minimum adalah 1

Damage dasar
↓
Efektivitas elemen
↓
Critical Hit
↓
Kurangi Defense
↓
Pastikan minimal 1

```
using UnityEngine;

public enum ElementType
{
    Fire,
    Water,
    Earth
}

public class DamageCalculator
{
    public static int CalculateDamage(
        int baseDamage,
        ElementType attacker,
        ElementType defender,
        int defense)
    {
        float damage = baseDamage;

        // Efektivitas elemen
        if (attacker == ElementType.Fire &&
            defender == ElementType.Earth)
        {
            damage *= 1.5f;
        }
        else if (attacker == ElementType.Earth &&
                 defender == ElementType.Water)
        {
            damage *= 1.5f;
        }
        else if (attacker == ElementType.Water &&
                 defender == ElementType.Fire)
        {
            damage *= 1.5f;
        }
        else if (attacker != defender)
        {
            damage *= 0.5f;
        }

        // Critical Hit 20%
        if (Random.value < 0.20f)
        {
            damage *= 2f;
        }

        // Defense dikurangi setelah semua multiplier
        damage -= defense;

        // Minimal damage 1
        damage = Mathf.Max(1, damage);

        return Mathf.RoundToInt(damage);
    }
}
```

CONTOH PERHITUNGAN DAMAGE

Diketahui:

Damage dasar = 40
Penyerang = Fire
Target = Earth
Critical = terjadi
Defense = 25

Perhitungan:

40 × 1,5 = 60
60 × 2 = 120
120 – 25 = 95
Damage akhir = 95

Game Characters & Players

```cs
using UnityEngine;

public class Player : GameCharacter
{
    public int experience = 0;
    public int level = 1;

    public override void Attack()
    {
        Debug.Log(characterName + " menyerang menggunakan senjata!");
    }
}
```

```cs
using UnityEngine;

public abstract class GameCharacter : MonoBehaviour
{
    public string characterName;
    public int maxHP = 100;
    protected int currentHP;

    protected virtual void Start()
    {
        currentHP = maxHP;
    }

    public void TakeDamage(int damage)
    {
        currentHP -= damage;

        if (currentHP < 0)
        {
            currentHP = 0;
        }
    }

    public bool IsAlive()
    {
        return currentHP > 0;
    }

    public abstract void Attack();
}
```

Enemy Spawner

```cs
using System.Collections.Generic;
using UnityEngine;

public class EnemySpawner : MonoBehaviour
{
    public GameObject player;

    private Rigidbody playerRb;
    private List<EnemyAI> enemyAIList = new List<EnemyAI>();

    void Start()
    {
        // Cari Player sekali
        player = GameObject.Find("Player");

        if (player != null)
        {
            playerRb = player.GetComponent<Rigidbody>();
        }

        // Cari Enemy sekali
        GameObject[] enemies =
            GameObject.FindGameObjectsWithTag("Enemy");

        foreach (GameObject enemy in enemies)
        {
            EnemyAI ai = enemy.GetComponent<EnemyAI>();

            if (ai != null)
            {
                enemyAIList.Add(ai);
            }
        }
    }

    void Update()
    {
        if (player == null || playerRb == null)
            return;

        foreach (EnemyAI enemyAI in enemyAIList)
        {
            float distance = Vector3.Distance(
                enemyAI.transform.position,
                player.transform.position
            );

            if (distance < 5f)
            {
                enemyAI.ChasePlayer(playerRb);
            }
        }
    }
}
```